Taller 1 – Búsqueda en Laberintos

Introducción

La resolución de laberintos es un problema clásico dentro del área de la Inteligencia Artificial y la teoría de grafos, ya que puede modelarse como un problema de búsqueda en un espacio de estados.

El objetivo de este trabajo consiste en implementar y analizar diferentes algoritmos de búsqueda, tanto no informados como informados, para encontrar un camino desde un nodo inicial hasta un nodo objetivo dentro de un laberinto generado automáticamente.

Se evaluaron los algoritmos DFS, BFS, Dijkstra y A∗, comparando su desempeño en términos de tiempo de ejecución, cantidad de nodos explorados y calidad de la solución obtenida.

Planteamiento del problema

El laberinto se representa como una matriz bidimensional donde cada celda puede ser transitable o no transitable (pared). A partir de esta representación, el problema se transforma en un grafo donde:

- Cada celda libre corresponde a un nodo
- Las conexiones entre celdas adyacentes corresponden a aristas
- Existe un nodo inicial (origen) y un nodo final (meta)

El objetivo es encontrar una ruta válida que conecte el nodo inicial con el nodo objetivo, optimizando criterios como el costo o la longitud del camino.

Marco teórico

Búsqueda en grafos

Los algoritmos de búsqueda en grafos permiten explorar un conjunto de estados para encontrar una solución. Estos algoritmos pueden clasificarse en:

- **Búsqueda no informada**: no utilizan información adicional sobre la meta

- **Búsqueda informada**: utilizan heurísticas para guiar la exploración

Algoritmos implementados

▼ DFS (Depth First Search)

El algoritmo DFS explora el grafo en profundidad, avanzando por un camino hasta que no puede continuar y luego retrocede.

Características:

- Implementado mediante pila (stack)
- Bajo consumo de memoria
- No garantiza encontrar el camino más corto

Complejidad:

- Tiempo: O(V + E)
- Puede ser ineficiente en grafos grandes

▼ BFS (Breadth First Search)

El algoritmo BFS recorre el grafo por niveles, explorando todos los vecinos antes de avanzar a niveles más profundos.

Características:

- Implementado mediante cola (queue)
- Garantiza el camino más corto en grafos no ponderados

Complejidad:

- Tiempo: O(V + E)
- Mayor consumo de memoria que DFS

▼ Dijkstra

El algoritmo de Dijkstra encuentra el camino de menor costo desde un nodo origen hacia todos los demás nodos del grafo.

Características:

- Utiliza una cola de prioridad
- Garantiza el camino óptimo
- En este problema se comporta similar a BFS debido a costos uniformes

Complejidad:

- Tiempo: O((V + E) log V)

▼ A∗ (A estrella)

El algoritmo A* es una extensión de Dijkstra que utiliza una heurística para estimar la distancia restante hasta la meta.

Se empleó la distancia Manhattan como heurística:

```
h(n) = |x1 - x2| + |y1 - y2|
```

Características:

- Combina costo real + estimación heurística
- Reduce significativamente la exploración

Complejidad:

- Depende de la calidad de la heurística
- En general más eficiente que Dijkstra

Metodología

Para el desarrollo del trabajo se siguieron los siguientes pasos:

1. Generación automática del laberinto
2. Conversión del laberinto en un grafo
3. Implementación de los algoritmos de búsqueda
4. Ejecución de cada algoritmo sobre el mismo laberinto
5. Recolección de métricas de desempeño

Métricas evaluadas

Se consideraron las siguientes métricas:

- **Tiempo de ejecución:** duración del algoritmo
- **Nodos visitados:** cantidad de estados explorados
- **Longitud del camino:** número de pasos hasta la meta

Resultados

Algoritmo	Tiempo (s)	Nodos visitados	Longitud del camino
DFS	0.20717	17881	1983
BFS	0.03881	20737	675
Dijkstra	0.04902	20737	675
A*	0.04751	19358	689

El comportamiento de los algoritmos muestra diferencias claras en eficiencia y calidad de solución.

El algoritmo **DFS** presenta el peor desempeño general. Aunque visitó menos nodos que BFS y Dijkstra, el camino obtenido fue extremadamente largo (1983), lo que confirma que no garantiza soluciones óptimas. Esto se debe a que DFS explora en profundidad sin considerar la cercanía a la meta.

El algoritmo **BFS** fue el más rápido (0.03881 s) y encontró el camino más corto (675 pasos). Sin embargo, exploró la mayor cantidad de nodos (20737), lo que implica un mayor costo computacional en términos de exploración.

El algoritmo **Dijkstra** presentó resultados prácticamente idénticos a BFS tanto en nodos visitados como en longitud del camino. Esto ocurre porque el problema no tiene pesos diferenciados, por lo que ambos algoritmos se comportan de manera equivalente.

El algoritmo **A*** logró reducir el número de nodos explorados (19358) en comparación con BFS y Dijkstra, manteniendo una longitud de camino cercana a la óptima (689). Esto evidencia la efectividad de la heurística para guiar la búsqueda hacia la meta.

En general, A* logra un mejor equilibrio entre eficiencia y calidad de solución, mientras que BFS garantiza optimalidad al costo de explorar más nodos.

Discusión

Los resultados evidencian que la utilización de heurísticas mejora considerablemente la eficiencia de los algoritmos de búsqueda.

Mientras BFS y Dijkstra garantizan soluciones óptimas, su costo en exploración es elevado. En contraste, A* logra un equilibrio entre optimalidad y eficiencia, explorando menos nodos gracias a la heurística de distancia Manhattan.

DFS, aunque sencillo, no es adecuado para este tipo de problemas cuando se busca eficiencia o calidad en la solución.

Resolución de preguntas

1. ¿Qué algoritmo fue el más eficiente en términos de tiempo?

El algoritmo más eficiente en términos de tiempo fue **BFS**, con un tiempo de ejecución de 0.03881 segundos.

Sin embargo, A* presentó un tiempo cercano, con la ventaja de explorar menos nodos, lo que lo hace más eficiente en términos generales.

---

2. ¿Cuál visitó la menor cantidad de nodos? ¿Por qué?

El algoritmo que visitó menos nodos fue **DFS (17881)**.

Sin embargo, esto no significa que sea mejor, ya que encontró un camino muy largo.

Esto ocurre porque DFS explora profundamente sin un criterio que lo acerque a la meta.

Por otro lado, entre los algoritmos eficientes, **A*** visitó menos nodos que BFS y Dijkstra, gracias al uso de una heurística.

---

3. ¿Hubo diferencia entre BFS y A*?

Sí hubo diferencia:

- BFS: 675 pasos
- A∗: 689 pasos

BFS encontró el camino más corto, mientras que A* encontró un camino ligeramente más largo.

Esto ocurre porque la heurística de A* guía la búsqueda y puede sacrificar ligeramente la optimalidad para mejorar la eficiencia.

---

4. ¿Dijkstra vs BFS?

No hubo diferencias en este caso:

- Ambos encontraron el mismo camino (675)
- Ambos exploraron el mismo número de nodos (20737)

Esto se debe a que todos los costos en el grafo son iguales, por lo que Dijkstra se comporta como BFS.

---

5. ¿Dijkstra vs A*?

A* es más eficiente que Dijkstra.

Aunque Dijkstra garantiza el camino óptimo, explora más nodos.

A* utiliza una heurística que le permite reducir el espacio de búsqueda, logrando un rendimiento superior.

---

6. ¿Cómo afecta la densidad de paredes al DFS?

A mayor densidad de paredes:

- DFS se vuelve más ineficiente
- Aumenta la probabilidad de explorar caminos sin salida
- Incrementa el tiempo de ejecución

Esto se debe a que DFS no tiene un criterio que lo guíe hacia la meta.